🎓 EduFlow

Student Performance & Attendance Analytics System

Tech Stack: Spring Boot (Java) + React.js + MySQL

SECTION 1: PROJECT OVERVIEW & FOLDER STRUCTURE

1.1 What We Are Building

EduFlow is a full-stack web application with:

- Backend: Spring Boot (Java) running on port 8080
- Frontend: React.js running on port 5173
- Database: MySQL
- Authentication: Email OTP + BCrypt password encryption
- PDF Generation: OpenPDF library

1.2 Backend Folder Structure (Eclipse)

Inside src/main/java/com/eduflow/

```
com.eduflow
├── controller/
│   ├── AuthController.java
│   ├── StudentController.java
│   ├── TeacherController.java
│   ├── AttendanceController.java
│   ├── MarksController.java
│   ├── AnalyticsController.java
│   ├── PdfController.java
│   └── AdminController.java
├── service/
│   ├── AuthService.java
│   ├── StudentService.java
│   ├── TeacherService.java
│   ├── AttendanceService.java
│   ├── MarksService.java
│   ├── AnalyticsService.java
│   ├── PdfService.java
│   ├── EmailService.java
│   └── OtpService.java
├── repository/
│   ├── UserRepository.java
│   ├── StudentRepository.java
│   ├── TeacherRepository.java
│   ├── AttendanceRepository.java
│   ├── MarksRepository.java
│   ├── OtpRepository.java
│   └── CourseRepository.java
```

```
├── model/
│   ├── User.java
│   ├── Student.java
│   ├── Teacher.java
│   ├── Course.java
│   ├── Attendance.java
│   ├── Marks.java
│   └── OtpEntity.java
├── dto/
│   ├── UserDTO.java
│   ├── StudentDTO.java
│   ├── TeacherDTO.java
│   ├── AttendanceDTO.java
│   ├── MarksDTO.java
│   ├── LoginRequest.java
│   ├── OtpRequest.java
│   └── AnalyticsDTO.java
├── config/
│   ├── CorsConfig.java
│   ├── SecurityConfig.java
│   └── WebConfig.java
└── exception/
    └── GlobalExceptionHandler.java
```

1.3 Frontend Folder Structure (VS Code)

```
eduflow-frontend/
├── src/
│   ├── pages/
│   │   ├── Login.jsx
│   │   ├── Signup.jsx
│   │   ├── OtpVerify.jsx
│   │   ├── ForgotPassword.jsx
│   │   ├── AdminDashboard.jsx
│   │   ├── TeacherDashboard.jsx
│   │   ├── StudentDashboard.jsx
│   │   ├── Students.jsx
│   │   ├── Teachers.jsx
│   │   ├── Attendance.jsx
│   │   ├── Marks.jsx
│   │   └── Analytics.jsx
│   ├── components/
│   │   ├── Sidebar.jsx
│   │   ├── Navbar.jsx
│   │   ├── StatCard.jsx
│   │   ├── Spinner.jsx
│   │   ├── ProtectedRoute.jsx
│   │   └── ErrorMessage.jsx
│   ├── api/
│   │   └── axiosConfig.js
│   ├── context/
│   │   └── AuthContext.jsx
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
├── .env
└── package.json
```

SECTION 2: SPRING BOOT BACKEND SETUP

2.1 Create Project at start.spring.io

Go to https://start.spring.io and set:

- Project: Maven
- Language: Java
- Spring Boot: 3.2.x
- Group: com.eduflow
- Artifact: eduflow
- Packaging: Jar
- Java: 17

Add these dependencies:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Spring Security
- Lombok
- Validation
- Spring Boot DevTools
- Java Mail Sender

2.2 pom.xml - Add OpenPDF for PDF generation

File: pom.xml (add inside <dependencies>)

```
<dependency>
    <groupId>com.github.librepdf</groupId>
    <artifactId>openpdf</artifactId>
    <version>1.3.30</version>
</dependency>
```

2.3 application.properties

File: src/main/resources/application.properties

```
# Server
server.port=8080

# Database
spring.datasource.url=jdbc:mysql://localhost:3306/eduflow_db
spring.datasource.username=root
spring.datasource.password=yourpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

# Email (Gmail)
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=youremail@gmail.com
```

```
spring.mail.password=your_app_password
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

# File Upload
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB
app.upload.dir=uploads/

# App Secret for OTP
app.otp.expiry-minutes=5
```

SECTION 3: MYSQL DATABASE & MODEL ENTITIES

3.1 Create MySQL Database

```
-- Run this in MySQL Workbench
CREATE DATABASE eduflow_db;
USE eduflow_db;
```

3.2 User.java

File: src/main/java/com/eduflow/model/User.java

```
package com.eduflow.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role;

    @Column(nullable = false)
    private boolean enabled = false;

    @Column(name = "profile_image")
    private String profileImage;

    public enum Role {
        ADMIN, TEACHER, STUDENT
    }
}
```

3.3 Student.java

File: src/main/java/com/eduflow/model/Student.java

```
package com.eduflow.model;
```

```java
import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "students")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne
    @JoinColumn(name = "user_id", unique = true)
    private User user;

    @Column(name = "roll_number")
    private String rollNumber;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    private String phone;
    private String address;
    private String gender;
    private String dateOfBirth;
}
```

3.4 Teacher.java

File: src/main/java/com/eduflow/model/Teacher.java

```java
package com.eduflow.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "teachers")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Teacher {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne
    @JoinColumn(name = "user_id", unique = true)
    private User user;

    private String subject;
    private String phone;
    private String qualification;
}
```

3.5 Course.java

File: src/main/java/com/eduflow/model/Course.java

```
package com.eduflow.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "courses")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String name;

    private String description;
    private int duration; // in months
}
```

3.6 Attendance.java

File: src/main/java/com/eduflow/model/Attendance.java

```
package com.eduflow.model;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDate;

@Entity
@Table(name = "attendance")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Attendance {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "student_id")
    private Student student;

    private LocalDate date;
    private String status; // "PRESENT" or "ABSENT"
    private String subject;
}
```

3.7 Marks.java

File: src/main/java/com/eduflow/model/Marks.java

```
package com.eduflow.model;
```

```
import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "marks")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Marks {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "student_id")
    private Student student;

    private String subject;
    private int marksObtained;
    private int totalMarks;
    private String examType; // "MID_TERM", "FINAL"
    private String semester;
}
```

3.8 OtpEntity.java

File: src/main/java/com/eduflow/model/OtpEntity.java

```
package com.eduflow.model;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "otp_table")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class OtpEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String email;

    @Column(nullable = false)
    private String otp;

    @Column(name = "expiry_time")
    private LocalDateTime expiryTime;
}
```

SECTION 4: DTO LAYER

DTOs (Data Transfer Objects) are used to send/receive data in APIs instead of directly using entities.

4.1 UserDTO.java

File: src/main/java/com/eduflow/dto/UserDTO.java

```
package com.eduflow.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserDTO {

    private Long id;

    @NotBlank(message = "Name is required")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Enter a valid email")
    private String email;

    @Size(min = 6, message = "Password must be at least 6 characters")
    private String password;

    private String role;
    private boolean enabled;
    private String profileImage;
}
```

4.2 StudentDTO.java

File: src/main/java/com/eduflow/dto/StudentDTO.java

```
package com.eduflow.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class StudentDTO {

    private Long id;

    @NotBlank(message = "Name is required")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Enter a valid email")
    private String email;
```

```
    private String rollNumber;
    private Long courseId;
    private String courseName;
    private String phone;
    private String address;
    private String gender;
    private String dateOfBirth;
    private String profileImage;
}
```

4.3 TeacherDTO.java

File: src/main/java/com/eduflow/dto/TeacherDTO.java

```
package com.eduflow.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class TeacherDTO {

    private Long id;

    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Enter a valid email")
    private String email;

    private String subject;
    private String phone;
    private String qualification;
}
```

4.4 AttendanceDTO.java

File: src/main/java/com/eduflow/dto/AttendanceDTO.java

```
package com.eduflow.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class AttendanceDTO {

    private Long id;

    @NotNull(message = "Student ID is required")
    private Long studentId;

    private String studentName;

    @NotBlank(message = "Date is required")
    private String date;
```

```
    @NotBlank(message = "Status is required")
    private String status;

    @NotBlank(message = "Subject is required")
    private String subject;
}
```

4.5 MarksDTO.java

File: src/main/java/com/eduflow/dto/MarksDTO.java

```
package com.eduflow.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class MarksDTO {

    private Long id;

    @NotNull(message = "Student ID is required")
    private Long studentId;

    private String studentName;

    @NotBlank(message = "Subject is required")
    private String subject;

    @Min(value = 0, message = "Marks cannot be negative")
    private int marksObtained;

    @Min(value = 1, message = "Total marks must be at least 1")
    private int totalMarks;

    private String examType;
    private String semester;
}
```

4.6 LoginRequest.java

File: src/main/java/com/eduflow/dto/LoginRequest.java

```
package com.eduflow.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class LoginRequest {

    @Email(message = "Enter valid email")
    @NotBlank(message = "Email is required")
    private String email;

    @NotBlank(message = "Password is required")
```

```
    private String password;
}
```

4.7 OtpRequest.java

File: src/main/java/com/eduflow/dto/OtpRequest.java

```
package com.eduflow.dto;

import jakarta.validation.constraints.*;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class OtpRequest {

    @Email
    @NotBlank
    private String email;

    private String otp;
    private String newPassword;
}
```

4.8 AnalyticsDTO.java

File: src/main/java/com/eduflow/dto/AnalyticsDTO.java

```
package com.eduflow.dto;

import lombok.*;
import java.util.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class AnalyticsDTO {

    private Long studentId;
    private String studentName;
    private double attendancePercentage;
    private double averageMarks;
    private String performanceStatus; // "Good", "Average", "Poor"
    private Map<String, Integer> subjectMarks;
    private int totalPresent;
    private int totalAbsent;
    private int totalClasses;
}
```

SECTION 5: REPOSITORIES

5.1 UserRepository.java

File: src/main/java/com/eduflow/repository/UserRepository.java

```
package com.eduflow.repository;

import com.eduflow.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

5.2 StudentRepository.java

File: src/main/java/com/eduflow/repository/StudentRepository.java

```
package com.eduflow.repository;

import com.eduflow.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;

public interface StudentRepository extends JpaRepository<Student, Long> {
    List<Student> findByCourseId(Long courseId);
    Optional<Student> findByUserId(Long userId);
    List<Student> findByUserNameContainingIgnoreCase(String name);
}
```

5.3 TeacherRepository.java

File: src/main/java/com/eduflow/repository/TeacherRepository.java

```
package com.eduflow.repository;

import com.eduflow.model.Teacher;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface TeacherRepository extends JpaRepository<Teacher, Long> {
    Optional<Teacher> findByUserId(Long userId);
}
```

5.4 AttendanceRepository.java

File: src/main/java/com/eduflow/repository/AttendanceRepository.java

```
package com.eduflow.repository;

import com.eduflow.model.Attendance;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.util.List;
```

```
public interface AttendanceRepository extends JpaRepository<Attendance, Long> {

    List<Attendance> findByStudentId(Long studentId);

    List<Attendance> findByStudentIdAndSubject(Long studentId, String subject);

    @Query("SELECT COUNT(a) FROM Attendance a WHERE a.student.id = :studentId AND
a.status = :status")
    long countByStudentIdAndStatus(Long studentId, String status);

    @Query("SELECT COUNT(a) FROM Attendance a WHERE a.student.id = :studentId")
    long countByStudentId(Long studentId);
}
```

5.5 MarksRepository.java

File: src/main/java/com/eduflow/repository/MarksRepository.java

```
package com.eduflow.repository;

import com.eduflow.model.Marks;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface MarksRepository extends JpaRepository<Marks, Long> {
    List<Marks> findByStudentId(Long studentId);
    List<Marks> findByStudentIdAndSubject(Long studentId, String subject);
    List<Marks> findByStudentIdAndSemester(Long studentId, String semester);
}
```

5.6 OtpRepository.java

File: src/main/java/com/eduflow/repository/OtpRepository.java

```
package com.eduflow.repository;

import com.eduflow.model.OtpEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface OtpRepository extends JpaRepository<OtpEntity, Long> {
    Optional<OtpEntity> findByEmail(String email);
    void deleteByEmail(String email);
}
```

5.7 CourseRepository.java

File: src/main/java/com/eduflow/repository/CourseRepository.java

```
package com.eduflow.repository;

import com.eduflow.model.Course;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CourseRepository extends JpaRepository<Course, Long> {
    boolean existsByName(String name);
}
```

SECTION 6: CONFIGURATION FILES

6.1 CorsConfig.java

File: src/main/java/com/eduflow/config/CorsConfig.java

```
package com.eduflow.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.*;
import org.springframework.web.filter.CorsFilter;

@Configuration
public class CorsConfig {

    @Bean
    public CorsFilter corsFilter() {
        CorsConfiguration config = new CorsConfiguration();
        config.addAllowedOrigin("http://localhost:5173");
        config.addAllowedOrigin("https://your-vercel-url.vercel.app");
        config.addAllowedMethod("*");
        config.addAllowedHeader("*");
        config.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return new CorsFilter(source);
    }
}
```

6.2 SecurityConfig.java

File: src/main/java/com/eduflow/config/SecurityConfig.java

```
package com.eduflow.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
```

```
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/uploads/**").permitAll()
                .anyRequest().permitAll()
            );
        return http.build();
    }
}
```

SECTION 7: GLOBAL EXCEPTION HANDLER

7.1 GlobalExceptionHandler.java

File: src/main/java/com/eduflow/exception/GlobalExceptionHandler.java

```java
package com.eduflow.exception;

import org.springframework.http.*;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.*;
import org.springframework.web.bind.annotation.*;
import java.util.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // Handles @Valid validation errors
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidation(
            MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errors.put(error.getField(), error.getDefaultMessage());
        }
        return ResponseEntity.badRequest().body(errors);
    }

    // Handles Runtime exceptions (user not found, invalid OTP etc)
    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<Map<String, String>> handleRuntime(RuntimeException ex) {
        Map<String, String> error = new HashMap<>();
        error.put("error", ex.getMessage());
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }

    // Handles all other exceptions
    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, String>> handleGeneral(Exception ex) {
        Map<String, String> error = new HashMap<>();
        error.put("error", "Something went wrong: " + ex.getMessage());
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

SECTION 8: OTP & EMAIL SERVICES

8.1 EmailService.java

File: src/main/java/com/eduflow/service/EmailService.java

```
package com.eduflow.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
public class EmailService {

    @Autowired
    private JavaMailSender mailSender;

    public void sendOtpEmail(String toEmail, String otp) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(toEmail);
        message.setSubject("EduFlow - Your OTP Code");
        message.setText(
            "Hello!\n\n" +
            "Your OTP for EduFlow is: " + otp + "\n\n" +
            "This OTP is valid for 5 minutes.\n\n" +
            "Do not share this OTP with anyone.\n\n" +
            "Team EduFlow"
        );
        mailSender.send(message);
    }
}
```

8.2 OtpService.java

File: src/main/java/com/eduflow/service/OtpService.java

```
package com.eduflow.service;

import com.eduflow.model.OtpEntity;
import com.eduflow.repository.OtpRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;
import java.util.Random;

@Service
public class OtpService {

    @Autowired
    private OtpRepository otpRepository;

    @Autowired
    private EmailService emailService;

    // Generate and send OTP
    @Transactional
    public void generateAndSendOtp(String email) {
```

```
        // Delete old OTP if exists
        otpRepository.deleteByEmail(email);

        // Generate 6-digit OTP
        String otp = String.format("%06d", new Random().nextInt(999999));

        // Set expiry to 5 minutes from now
        LocalDateTime expiry = LocalDateTime.now().plusMinutes(5);

        // Save OTP
        OtpEntity otpEntity = new OtpEntity(null, email, otp, expiry);
        otpRepository.save(otpEntity);

        // Send email
        emailService.sendOtpEmail(email, otp);
    }

    // Verify OTP
    @Transactional
    public boolean verifyOtp(String email, String otp) {
        OtpEntity otpEntity = otpRepository.findByEmail(email)
            .orElseThrow(() -> new RuntimeException("OTP not found. Please request a
new one."));

        // Check if expired
        if (LocalDateTime.now().isAfter(otpEntity.getExpiryTime())) {
            otpRepository.deleteByEmail(email);
            throw new RuntimeException("OTP has expired. Please request a new one.");
        }

        // Check if OTP matches
        if (!otpEntity.getOtp().equals(otp)) {
            throw new RuntimeException("Invalid OTP. Please try again.");
        }

        // Delete OTP after successful verification
        otpRepository.deleteByEmail(email);
        return true;
    }
}
```

SECTION 9: AUTHENTICATION - SERVICE & CONTROLLER

9.1 AuthService.java

File: src/main/java/com/eduflow/service/AuthService.java

```
package com.eduflow.service;

import com.eduflow.dto.*;
import com.eduflow.model.*;
import com.eduflow.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import java.util.*;

@Service
public class AuthService {

    @Autowired private UserRepository userRepository;
    @Autowired private OtpService otpService;
    @Autowired private PasswordEncoder passwordEncoder;

    // Step 1: Send OTP to email before registration
    public Map<String, String> sendRegistrationOtp(String email) {
        if (userRepository.existsByEmail(email)) {
            throw new RuntimeException("Email already registered.");
        }
        otpService.generateAndSendOtp(email);
        Map<String, String> res = new HashMap<>();
        res.put("message", "OTP sent to " + email);
        return res;
    }

    // Step 2: Verify OTP and register user
    public Map<String, String> registerUser(UserDTO dto, String otp) {
        otpService.verifyOtp(dto.getEmail(), otp);

        User user = new User();
        user.setName(dto.getName());
        user.setEmail(dto.getEmail());
        user.setPassword(passwordEncoder.encode(dto.getPassword()));
        user.setRole(User.Role.valueOf(dto.getRole()));
        user.setEnabled(true);
        userRepository.save(user);

        Map<String, String> res = new HashMap<>();
        res.put("message", "Registration successful!");
        return res;
    }

    // Login
    public Map<String, Object> login(LoginRequest request) {
        User user = userRepository.findByEmail(request.getEmail())
            .orElseThrow(() -> new RuntimeException("User not found."));

        if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
            throw new RuntimeException("Invalid password.");
        }

        Map<String, Object> res = new HashMap<>();
```

```
        res.put("id", user.getId());
        res.put("name", user.getName());
        res.put("email", user.getEmail());
        res.put("role", user.getRole());
        res.put("message", "Login successful!");
        return res;
    }

    // Send OTP for Forgot Password
    public Map<String, String> sendForgotPasswordOtp(String email) {
        if (!userRepository.existsByEmail(email)) {
            throw new RuntimeException("No account found with this email.");
        }
        otpService.generateAndSendOtp(email);
        Map<String, String> res = new HashMap<>();
        res.put("message", "OTP sent to " + email);
        return res;
    }

    // Reset Password after OTP verification
    public Map<String, String> resetPassword(OtpRequest request) {
        otpService.verifyOtp(request.getEmail(), request.getOtp());

        User user = userRepository.findByEmail(request.getEmail())
            .orElseThrow(() -> new RuntimeException("User not found."));

        user.setPassword(passwordEncoder.encode(request.getNewPassword()));
        userRepository.save(user);

        Map<String, String> res = new HashMap<>();
        res.put("message", "Password reset successful!");
        return res;
    }
}
```

9.2 AuthController.java

File: src/main/java/com/eduflow/controller/AuthController.java

```
package com.eduflow.controller;

import com.eduflow.dto.*;
import com.eduflow.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthService authService;

    // POST /api/auth/send-otp?email=user@gmail.com
    @PostMapping("/send-otp")
    public ResponseEntity<?> sendOtp(@RequestParam String email) {
        return ResponseEntity.ok(authService.sendRegistrationOtp(email));
    }

    // POST /api/auth/register?otp=123456
```

```
    @PostMapping("/register")
    public ResponseEntity<?> register(
            @Valid @RequestBody UserDTO dto,
            @RequestParam String otp) {
        return ResponseEntity.ok(authService.registerUser(dto, otp));
    }

    // POST /api/auth/login
    @PostMapping("/login")
    public ResponseEntity<?> login(@Valid @RequestBody LoginRequest request) {
        return ResponseEntity.ok(authService.login(request));
    }

    // POST /api/auth/forgot-password?email=user@gmail.com
    @PostMapping("/forgot-password")
    public ResponseEntity<?> forgotPassword(@RequestParam String email) {
        return ResponseEntity.ok(authService.sendForgotPasswordOtp(email));
    }

    // POST /api/auth/reset-password
    @PostMapping("/reset-password")
    public ResponseEntity<?> resetPassword(@RequestBody OtpRequest request) {
        return ResponseEntity.ok(authService.resetPassword(request));
    }
}
```

SECTION 10: STUDENT SERVICE & CONTROLLER

10.1 StudentService.java

File: src/main/java/com/eduflow/service/StudentService.java

```
package com.eduflow.service;

import com.eduflow.dto.StudentDTO;
import com.eduflow.model.*;
import com.eduflow.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import java.util.*;
import java.util.stream.Collectors;

@Service
public class StudentService {

    @Autowired private StudentRepository studentRepository;
    @Autowired private UserRepository userRepository;
    @Autowired private CourseRepository courseRepository;
    @Autowired private PasswordEncoder passwordEncoder;

    // Add student (also creates a user account)
    public StudentDTO addStudent(StudentDTO dto) {
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new RuntimeException("Email already exists.");
        }
        // Create user
        User user = new User();
        user.setName(dto.getName());
        user.setEmail(dto.getEmail());
        user.setPassword(passwordEncoder.encode("student@123")); // default password
        user.setRole(User.Role.STUDENT);
        user.setEnabled(true);
        userRepository.save(user);

        // Create student
        Student student = new Student();
        student.setUser(user);
        student.setRollNumber(dto.getRollNumber());
        student.setPhone(dto.getPhone());
        student.setAddress(dto.getAddress());
        student.setGender(dto.getGender());
        student.setDateOfBirth(dto.getDateOfBirth());

        if (dto.getCourseId() != null) {
            Course course = courseRepository.findById(dto.getCourseId())
                .orElseThrow(() -> new RuntimeException("Course not found."));
            student.setCourse(course);
        }
        studentRepository.save(student);
        return toDTO(student);
    }

    // Get all students
    public List<StudentDTO> getAllStudents() {
        return studentRepository.findAll()
            .stream().map(this::toDTO).collect(Collectors.toList());
    }
```

```java
    // Get single student
    public StudentDTO getStudentById(Long id) {
        Student student = studentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Student not found."));
        return toDTO(student);
    }

    // Update student
    public StudentDTO updateStudent(Long id, StudentDTO dto) {
        Student student = studentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Student not found."));

        student.getUser().setName(dto.getName());
        student.setPhone(dto.getPhone());
        student.setAddress(dto.getAddress());
        student.setGender(dto.getGender());
        student.setDateOfBirth(dto.getDateOfBirth());
        student.setRollNumber(dto.getRollNumber());

        if (dto.getCourseId() != null) {
            Course course = courseRepository.findById(dto.getCourseId())
                .orElseThrow(() -> new RuntimeException("Course not found."));
            student.setCourse(course);
        }
        studentRepository.save(student);
        return toDTO(student);
    }

    // Delete student
    public Map<String, String> deleteStudent(Long id) {
        studentRepository.deleteById(id);
        Map<String, String> res = new HashMap<>();
        res.put("message", "Student deleted successfully.");
        return res;
    }

    // Search by name
    public List<StudentDTO> searchByName(String name) {
        return studentRepository.findByUserNameContainingIgnoreCase(name)
            .stream().map(this::toDTO).collect(Collectors.toList());
    }

    // Convert Entity to DTO
    public StudentDTO toDTO(Student s) {
        StudentDTO dto = new StudentDTO();
        dto.setId(s.getId());
        dto.setName(s.getUser().getName());
        dto.setEmail(s.getUser().getEmail());
        dto.setRollNumber(s.getRollNumber());
        dto.setPhone(s.getPhone());
        dto.setAddress(s.getAddress());
        dto.setGender(s.getGender());
        dto.setDateOfBirth(s.getDateOfBirth());
        dto.setProfileImage(s.getUser().getProfileImage());
        if (s.getCourse() != null) {
            dto.setCourseId(s.getCourse().getId());
            dto.setCourseName(s.getCourse().getName());
        }
        return dto;
    }
}
```

10.2 StudentController.java

File: src/main/java/com/eduflow/controller/StudentController.java

```java
package com.eduflow.controller;

import com.eduflow.dto.StudentDTO;
import com.eduflow.service.StudentService;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    @Autowired
    private StudentService studentService;

    @PostMapping
    public ResponseEntity<?> add(@Valid @RequestBody StudentDTO dto) {
        return ResponseEntity.ok(studentService.addStudent(dto));
    }

    @GetMapping
    public ResponseEntity<?> getAll() {
        return ResponseEntity.ok(studentService.getAllStudents());
    }

    @GetMapping("/{id}")
    public ResponseEntity<?> getById(@PathVariable Long id) {
        return ResponseEntity.ok(studentService.getStudentById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id,
                                    @Valid @RequestBody StudentDTO dto) {
        return ResponseEntity.ok(studentService.updateStudent(id, dto));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> delete(@PathVariable Long id) {
        return ResponseEntity.ok(studentService.deleteStudent(id));
    }

    @GetMapping("/search")
    public ResponseEntity<?> search(@RequestParam String name) {
        return ResponseEntity.ok(studentService.searchByName(name));
    }
}
```

SECTION 11: ATTENDANCE & MARKS

11.1 AttendanceService.java

File: src/main/java/com/eduflow/service/AttendanceService.java

```
package com.eduflow.service;

import com.eduflow.dto.AttendanceDTO;
import com.eduflow.model.*;
import com.eduflow.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.time.LocalDate;
import java.util.*;
import java.util.stream.Collectors;

@Service
public class AttendanceService {

    @Autowired private AttendanceRepository attendanceRepository;
    @Autowired private StudentRepository studentRepository;

    public AttendanceDTO markAttendance(AttendanceDTO dto) {
        Student student = studentRepository.findById(dto.getStudentId())
            .orElseThrow(() -> new RuntimeException("Student not found."));

        Attendance att = new Attendance();
        att.setStudent(student);
        att.setDate(LocalDate.parse(dto.getDate()));
        att.setStatus(dto.getStatus());
        att.setSubject(dto.getSubject());
        attendanceRepository.save(att);
        return toDTO(att);
    }

    public List<AttendanceDTO> getStudentAttendance(Long studentId) {
        return attendanceRepository.findByStudentId(studentId)
            .stream().map(this::toDTO).collect(Collectors.toList());
    }

    public List<AttendanceDTO> getAllAttendance() {
        return attendanceRepository.findAll()
            .stream().map(this::toDTO).collect(Collectors.toList());
    }

    public Map<String, String> deleteAttendance(Long id) {
        attendanceRepository.deleteById(id);
        Map<String, String> res = new HashMap<>();
        res.put("message", "Deleted successfully.");
        return res;
    }

    private AttendanceDTO toDTO(Attendance a) {
        AttendanceDTO dto = new AttendanceDTO();
        dto.setId(a.getId());
        dto.setStudentId(a.getStudent().getId());
        dto.setStudentName(a.getStudent().getUser().getName());
        dto.setDate(a.getDate().toString());
        dto.setStatus(a.getStatus());
        dto.setSubject(a.getSubject());
```

```
        return dto;
    }
}
```

11.2 AttendanceController.java

File: src/main/java/com/eduflow/controller/AttendanceController.java

```
package com.eduflow.controller;

import com.eduflow.dto.AttendanceDTO;
import com.eduflow.service.AttendanceService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/attendance")
public class AttendanceController {

    @Autowired private AttendanceService attendanceService;

    @PostMapping
    public ResponseEntity<?> mark(@RequestBody AttendanceDTO dto) {
        return ResponseEntity.ok(attendanceService.markAttendance(dto));
    }

    @GetMapping("/student/{studentId}")
    public ResponseEntity<?> getByStudent(@PathVariable Long studentId) {
        return ResponseEntity.ok(attendanceService.getStudentAttendance(studentId));
    }

    @GetMapping
    public ResponseEntity<?> getAll() {
        return ResponseEntity.ok(attendanceService.getAllAttendance());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> delete(@PathVariable Long id) {
        return ResponseEntity.ok(attendanceService.deleteAttendance(id));
    }
}
```

11.3 MarksService.java

File: src/main/java/com/eduflow/service/MarksService.java

```
package com.eduflow.service;

import com.eduflow.dto.MarksDTO;
import com.eduflow.model.*;
import com.eduflow.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.*;
import java.util.stream.Collectors;

@Service
public class MarksService {

    @Autowired private MarksRepository marksRepository;
```

```java
    @Autowired private StudentRepository studentRepository;

    public MarksDTO addMarks(MarksDTO dto) {
        Student student = studentRepository.findById(dto.getStudentId())
            .orElseThrow(() -> new RuntimeException("Student not found."));

        Marks marks = new Marks();
        marks.setStudent(student);
        marks.setSubject(dto.getSubject());
        marks.setMarksObtained(dto.getMarksObtained());
        marks.setTotalMarks(dto.getTotalMarks());
        marks.setExamType(dto.getExamType());
        marks.setSemester(dto.getSemester());
        marksRepository.save(marks);
        return toDTO(marks);
    }

    public List<MarksDTO> getStudentMarks(Long studentId) {
        return marksRepository.findByStudentId(studentId)
            .stream().map(this::toDTO).collect(Collectors.toList());
    }

    public List<MarksDTO> getAllMarks() {
        return marksRepository.findAll()
            .stream().map(this::toDTO).collect(Collectors.toList());
    }

    public MarksDTO updateMarks(Long id, MarksDTO dto) {
        Marks marks = marksRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Marks not found."));
        marks.setMarksObtained(dto.getMarksObtained());
        marks.setTotalMarks(dto.getTotalMarks());
        marks.setSubject(dto.getSubject());
        marks.setExamType(dto.getExamType());
        marks.setSemester(dto.getSemester());
        marksRepository.save(marks);
        return toDTO(marks);
    }

    public Map<String, String> deleteMarks(Long id) {
        marksRepository.deleteById(id);
        Map<String, String> res = new HashMap<>();
        res.put("message", "Deleted successfully.");
        return res;
    }

    private MarksDTO toDTO(Marks m) {
        MarksDTO dto = new MarksDTO();
        dto.setId(m.getId());
        dto.setStudentId(m.getStudent().getId());
        dto.setStudentName(m.getStudent().getUser().getName());
        dto.setSubject(m.getSubject());
        dto.setMarksObtained(m.getMarksObtained());
        dto.setTotalMarks(m.getTotalMarks());
        dto.setExamType(m.getExamType());
        dto.setSemester(m.getSemester());
        return dto;
    }
}
```

11.4 MarksController.java

File: src/main/java/com/eduflow/controller/MarksController.java

```
package com.eduflow.controller;

import com.eduflow.dto.MarksDTO;
import com.eduflow.service.MarksService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/marks")
public class MarksController {

    @Autowired private MarksService marksService;

    @PostMapping
    public ResponseEntity<?> add(@RequestBody MarksDTO dto) {
        return ResponseEntity.ok(marksService.addMarks(dto));
    }

    @GetMapping("/student/{studentId}")
    public ResponseEntity<?> getByStudent(@PathVariable Long studentId) {
        return ResponseEntity.ok(marksService.getStudentMarks(studentId));
    }

    @GetMapping
    public ResponseEntity<?> getAll() {
        return ResponseEntity.ok(marksService.getAllMarks());
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody MarksDTO dto)
{
        return ResponseEntity.ok(marksService.updateMarks(id, dto));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> delete(@PathVariable Long id) {
        return ResponseEntity.ok(marksService.deleteMarks(id));
    }
}
```

SECTION 12: ANALYTICS SERVICE & CONTROLLER

12.1 AnalyticsService.java

File: src/main/java/com/eduflow/service/AnalyticsService.java

```
package com.eduflow.service;

import com.eduflow.dto.AnalyticsDTO;
import com.eduflow.model.*;
import com.eduflow.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.*;
import java.util.stream.Collectors;

@Service
public class AnalyticsService {

    @Autowired private StudentRepository studentRepository;
    @Autowired private AttendanceRepository attendanceRepository;
    @Autowired private MarksRepository marksRepository;

    public AnalyticsDTO getStudentAnalytics(Long studentId) {
        Student student = studentRepository.findById(studentId)
            .orElseThrow(() -> new RuntimeException("Student not found."));

        // Attendance calculation
        long total = attendanceRepository.countByStudentId(studentId);
        long present = attendanceRepository.countByStudentIdAndStatus(studentId,
"PRESENT");
        long absent = total - present;
        double attendancePct = total == 0 ? 0 : (present * 100.0) / total;

        // Marks calculation
        List<Marks> marksList = marksRepository.findByStudentId(studentId);
        Map<String, Integer> subjectMarks = new LinkedHashMap<>();
        for (Marks m : marksList) {
            subjectMarks.put(m.getSubject(), m.getMarksObtained());
        }

        double avgMarks = marksList.isEmpty() ? 0 :
            marksList.stream()
                .mapToDouble(m -> (m.getMarksObtained() * 100.0) / m.getTotalMarks())
                .average().orElse(0);

        // Performance status
        String status;
        if (avgMarks >= 75) status = "Good";
        else if (avgMarks >= 50) status = "Average";
        else status = "Poor";

        AnalyticsDTO dto = new AnalyticsDTO();
        dto.setStudentId(studentId);
        dto.setStudentName(student.getUser().getName());
        dto.setAttendancePercentage(Math.round(attendancePct * 10.0) / 10.0);
        dto.setAverageMarks(Math.round(avgMarks * 10.0) / 10.0);
        dto.setPerformanceStatus(status);
        dto.setSubjectMarks(subjectMarks);
        dto.setTotalPresent((int) present);
        dto.setTotalAbsent((int) absent);
```

```
        dto.setTotalClasses((int) total);
        return dto;
    }

    // Analytics for all students (admin dashboard)
    public Map<String, Object> getDashboardStats() {
        long totalStudents = studentRepository.count();
        Map<String, Object> stats = new HashMap<>();
        stats.put("totalStudents", totalStudents);
        stats.put("totalAttendanceRecords", attendanceRepository.count());
        stats.put("totalMarksRecords", marksRepository.count());
        return stats;
    }
}
```

12.2 AnalyticsController.java

File: src/main/java/com/eduflow/controller/AnalyticsController.java

```
package com.eduflow.controller;

import com.eduflow.service.AnalyticsService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/analytics")
public class AnalyticsController {

    @Autowired private AnalyticsService analyticsService;

    @GetMapping("/student/{studentId}")
    public ResponseEntity<?> getStudentAnalytics(@PathVariable Long studentId) {
        return ResponseEntity.ok(analyticsService.getStudentAnalytics(studentId));
    }

    @GetMapping("/dashboard")
    public ResponseEntity<?> getDashboard() {
        return ResponseEntity.ok(analyticsService.getDashboardStats());
    }
}
```

SECTION 13: PDF GENERATION

13.1 PdfService.java

File: src/main/java/com/eduflow/service/PdfService.java

```
package com.eduflow.service;

import com.eduflow.model.*;
import com.eduflow.repository.*;
import com.lowagie.text.*;
import com.lowagie.text.pdf.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.io.*;
import java.util.List;

@Service
public class PdfService {

    @Autowired private StudentRepository studentRepository;
    @Autowired private AttendanceRepository attendanceRepository;
    @Autowired private MarksRepository marksRepository;

    // Generate Marksheet PDF
    public byte[] generateMarksheet(Long studentId) throws Exception {
        Student student = studentRepository.findById(studentId)
            .orElseThrow(() -> new RuntimeException("Student not found."));
        List<Marks> marksList = marksRepository.findByStudentId(studentId);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        Document doc = new Document();
        PdfWriter.getInstance(doc, baos);
        doc.open();

        // Title
        Font titleFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 20);
        Paragraph title = new Paragraph("EduFlow - Student Marksheet", titleFont);
        title.setAlignment(Element.ALIGN_CENTER);
        doc.add(title);
        doc.add(new Paragraph(" "));

        // Student Info
        Font boldFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 12);
        Font normalFont = FontFactory.getFont(FontFactory.HELVETICA, 12);
        doc.add(new Paragraph("Name: " + student.getUser().getName(), boldFont));
        doc.add(new Paragraph("Email: " + student.getUser().getEmail(), normalFont));
        doc.add(new Paragraph("Roll No: " + student.getRollNumber(), normalFont));
        if (student.getCourse() != null)
            doc.add(new Paragraph("Course: " + student.getCourse().getName(),
normalFont));
        doc.add(new Paragraph(" "));

        // Marks Table
        PdfPTable table = new PdfPTable(4);
        table.setWidthPercentage(100);
        String[] headers = {"Subject", "Marks Obtained", "Total Marks", "Percentage"};
        for (String h : headers) {
            PdfPCell cell = new PdfPCell(new Phrase(h, boldFont));
            cell.setBackgroundColor(new java.awt.Color(107, 33, 168));
            cell.getPhrase().getFont().setColor(new java.awt.Color(255,255,255));
```

```
            table.addCell(cell);
        }
        double totalPct = 0;
        for (Marks m : marksList) {
            double pct = (m.getMarksObtained() * 100.0) / m.getTotalMarks();
            totalPct += pct;
            table.addCell(new Phrase(m.getSubject(), normalFont));
            table.addCell(new Phrase(String.valueOf(m.getMarksObtained()),
normalFont));
            table.addCell(new Phrase(String.valueOf(m.getTotalMarks()), normalFont));
            table.addCell(new Phrase(String.format("%.1f%%", pct), normalFont));
        }
        doc.add(table);
        doc.add(new Paragraph(" "));

        double avg = marksList.isEmpty() ? 0 : totalPct / marksList.size();
        doc.add(new Paragraph("Average Percentage: " + String.format("%.1f%%", avg),
boldFont));
        String status = avg >= 75 ? "Good" : avg >= 50 ? "Average" : "Poor";
        doc.add(new Paragraph("Performance: " + status, boldFont));

        doc.close();
        return baos.toByteArray();
    }

    // Generate Attendance PDF
    public byte[] generateAttendancePdf(Long studentId) throws Exception {
        Student student = studentRepository.findById(studentId)
            .orElseThrow(() -> new RuntimeException("Student not found."));
        List<Attendance> attList = attendanceRepository.findByStudentId(studentId);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        Document doc = new Document();
        PdfWriter.getInstance(doc, baos);
        doc.open();

        Font titleFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 20);
        Paragraph title = new Paragraph("EduFlow - Attendance Report", titleFont);
        title.setAlignment(Element.ALIGN_CENTER);
        doc.add(title);
        doc.add(new Paragraph(" "));

        Font boldFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 12);
        Font normalFont = FontFactory.getFont(FontFactory.HELVETICA, 12);
        doc.add(new Paragraph("Student: " + student.getUser().getName(), boldFont));
        doc.add(new Paragraph("Email: " + student.getUser().getEmail(), normalFont));
        doc.add(new Paragraph(" "));

        PdfPTable table = new PdfPTable(3);
        table.setWidthPercentage(100);
        for (String h : new String[]{"Date", "Subject", "Status"}) {
            PdfPCell cell = new PdfPCell(new Phrase(h, boldFont));
            cell.setBackgroundColor(new java.awt.Color(107, 33, 168));
            cell.getPhrase().getFont().setColor(new java.awt.Color(255,255,255));
            table.addCell(cell);
        }
        long present = 0;
        for (Attendance a : attList) {
            if ("PRESENT".equals(a.getStatus())) present++;
            table.addCell(new Phrase(a.getDate().toString(), normalFont));
            table.addCell(new Phrase(a.getSubject(), normalFont));
            table.addCell(new Phrase(a.getStatus(), normalFont));
        }
        doc.add(table);
```

```
        doc.add(new Paragraph(" "));

        double pct = attList.isEmpty() ? 0 : (present * 100.0) / attList.size();
        doc.add(new Paragraph("Attendance Percentage: " + String.format("%.1f%%",
pct), boldFont));

        doc.close();
        return baos.toByteArray();
    }
}
```

13.2 PdfController.java

File: src/main/java/com/eduflow/controller/PdfController.java

```
package com.eduflow.controller;

import com.eduflow.service.PdfService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/pdf")
public class PdfController {

    @Autowired private PdfService pdfService;

    @GetMapping("/marksheet/{studentId}")
    public ResponseEntity<byte[]> getMarksheet(@PathVariable Long studentId) throws
Exception {
        byte[] pdf = pdfService.generateMarksheet(studentId);
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_PDF);
        headers.setContentDisposition(ContentDisposition
            .attachment().filename("marksheet_" + studentId + ".pdf").build());
        return new ResponseEntity<>(pdf, headers, HttpStatus.OK);
    }

    @GetMapping("/attendance/{studentId}")
    public ResponseEntity<byte[]> getAttendance(@PathVariable Long studentId) throws
Exception {
        byte[] pdf = pdfService.generateAttendancePdf(studentId);
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_PDF);
        headers.setContentDisposition(ContentDisposition
            .attachment().filename("attendance_" + studentId + ".pdf").build());
        return new ResponseEntity<>(pdf, headers, HttpStatus.OK);
    }
}
```

SECTION 14: TEACHER & ADMIN CONTROLLERS

14.1 TeacherService.java

File: src/main/java/com/eduflow/service/TeacherService.java

```
package com.eduflow.service;

import com.eduflow.dto.TeacherDTO;
import com.eduflow.model.*;
import com.eduflow.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import java.util.*;
import java.util.stream.Collectors;

@Service
public class TeacherService {

    @Autowired private TeacherRepository teacherRepository;
    @Autowired private UserRepository userRepository;
    @Autowired private PasswordEncoder passwordEncoder;

    public TeacherDTO addTeacher(TeacherDTO dto) {
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new RuntimeException("Email already exists.");
        }
        User user = new User();
        user.setName(dto.getName());
        user.setEmail(dto.getEmail());
        user.setPassword(passwordEncoder.encode("teacher@123"));
        user.setRole(User.Role.TEACHER);
        user.setEnabled(true);
        userRepository.save(user);

        Teacher teacher = new Teacher();
        teacher.setUser(user);
        teacher.setSubject(dto.getSubject());
        teacher.setPhone(dto.getPhone());
        teacher.setQualification(dto.getQualification());
        teacherRepository.save(teacher);
        return toDTO(teacher);
    }

    public List<TeacherDTO> getAllTeachers() {
        return teacherRepository.findAll()
            .stream().map(this::toDTO).collect(Collectors.toList());
    }

    public Map<String, String> deleteTeacher(Long id) {
        teacherRepository.deleteById(id);
        Map<String, String> res = new HashMap<>();
        res.put("message", "Teacher deleted.");
        return res;
    }

    private TeacherDTO toDTO(Teacher t) {
        TeacherDTO dto = new TeacherDTO();
        dto.setId(t.getId());
        dto.setName(t.getUser().getName());
```

```
        dto.setEmail(t.getUser().getEmail());
        dto.setSubject(t.getSubject());
        dto.setPhone(t.getPhone());
        dto.setQualification(t.getQualification());
        return dto;
    }
}
```

14.2 TeacherController.java

File: src/main/java/com/eduflow/controller/TeacherController.java

```
package com.eduflow.controller;

import com.eduflow.dto.TeacherDTO;
import com.eduflow.service.TeacherService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/teachers")
public class TeacherController {

    @Autowired private TeacherService teacherService;

    @PostMapping
    public ResponseEntity<?> add(@RequestBody TeacherDTO dto) {
        return ResponseEntity.ok(teacherService.addTeacher(dto));
    }

    @GetMapping
    public ResponseEntity<?> getAll() {
        return ResponseEntity.ok(teacherService.getAllTeachers());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> delete(@PathVariable Long id) {
        return ResponseEntity.ok(teacherService.deleteTeacher(id));
    }
}
```

SECTION 15: REACT FRONTEND SETUP

15.1 Create React Project

Run these commands in VS Code terminal:

```
npm create vite@latest eduflow-frontend -- --template react
cd eduflow-frontend
npm install
npm install axios react-router-dom recharts react-hot-toast
npm install lucide-react
```

15.2 .env File

File: eduflow-frontend/.env

```
VITE_API_URL=http://localhost:8080
```

15.3 src/api/axiosConfig.js

```
import axios from "axios";

const API = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: false,
});

// Attach user info from localStorage to every request
API.interceptors.request.use((config) => {
  const user = JSON.parse(localStorage.getItem("eduflow_user") || "{}");
  if (user.id) {
    config.headers["X-User-Id"] = user.id;
    config.headers["X-User-Role"] = user.role;
  }
  return config;
});

export default API;
```

15.4 src/context/AuthContext.jsx

```
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(() => {
    const stored = localStorage.getItem("eduflow_user");
    return stored ? JSON.parse(stored) : null;
  });

  const login = (userData) => {
    localStorage.setItem("eduflow_user", JSON.stringify(userData));
    setUser(userData);
  };

  const logout = () => {
```

```
    localStorage.removeItem("eduflow_user");
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);
```

15.5 src/components/ProtectedRoute.jsx

```
import { Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

const ProtectedRoute = ({ children, roles }) => {
  const { user } = useAuth();

  if (!user) return <Navigate to="/login" replace />;

  if (roles && !roles.includes(user.role)) {
    return <Navigate to="/login" replace />;
  }

  return children;
};

export default ProtectedRoute;
```

15.6 src/components/Spinner.jsx

```
const Spinner = () => (
  <div style={{
    display: "flex", justifyContent: "center",
    alignItems: "center", height: "200px"
  }}>
    <div style={{
      width: "50px", height: "50px",
      border: "5px solid #f3f3f3",
      borderTop: "5px solid #7C3AED",
      borderRadius: "50%",
      animation: "spin 1s linear infinite"
    }} />
    <style>{`@keyframes spin { to { transform: rotate(360deg); } }`}</style>
  </div>
);

export default Spinner;
```

15.7 src/App.jsx

```
import { BrowserRouter as Router, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider, useAuth } from "./context/AuthContext";
import ProtectedRoute from "./components/ProtectedRoute";
import Login from "./pages/Login";
import Signup from "./pages/Signup";
```

```
import OtpVerify from "./pages/OtpVerify";
import ForgotPassword from "./pages/ForgotPassword";
import AdminDashboard from "./pages/AdminDashboard";
import TeacherDashboard from "./pages/TeacherDashboard";
import StudentDashboard from "./pages/StudentDashboard";
import Students from "./pages/Students";
import Teachers from "./pages/Teachers";
import Attendance from "./pages/Attendance";
import Marks from "./pages/Marks";
import Analytics from "./pages/Analytics";

const DashboardRouter = () => {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" />;
  if (user.role === "ADMIN") return <Navigate to="/admin" />;
  if (user.role === "TEACHER") return <Navigate to="/teacher" />;
  return <Navigate to="/student" />;
};

function App() {
  return (
    <AuthProvider>
      <Router>
        <Routes>
          <Route path="/" element={<DashboardRouter />} />
          <Route path="/login" element={<Login />} />
          <Route path="/signup" element={<Signup />} />
          <Route path="/verify-otp" element={<OtpVerify />} />
          <Route path="/forgot-password" element={<ForgotPassword />} />
          <Route path="/admin" element={
            <ProtectedRoute roles={["ADMIN"]}><AdminDashboard /></ProtectedRoute>} />
          <Route path="/teacher" element={
            <ProtectedRoute roles={["TEACHER"]}><TeacherDashboard /></ProtectedRoute>}
/>
          <Route path="/student" element={
            <ProtectedRoute roles={["STUDENT"]}><StudentDashboard /></ProtectedRoute>}
/>
          <Route path="/students" element={
            <ProtectedRoute roles={["ADMIN","TEACHER"]}><Students /></ProtectedRoute>}
/>
          <Route path="/teachers" element={
            <ProtectedRoute roles={["ADMIN"]}><Teachers /></ProtectedRoute>} />
          <Route path="/attendance" element={
            <ProtectedRoute roles={["ADMIN","TEACHER","STUDENT"]}><Attendance
/></ProtectedRoute>} />
          <Route path="/marks" element={
            <ProtectedRoute roles={["ADMIN","TEACHER","STUDENT"]}><Marks
/></ProtectedRoute>} />
          <Route path="/analytics" element={
            <ProtectedRoute roles={["ADMIN","TEACHER","STUDENT"]}><Analytics
/></ProtectedRoute>} />
        </Routes>
      </Router>
    </AuthProvider>
  );
}

export default App;
```

SECTION 16: GLOBAL CSS (PURPLE & WHITE THEME)

16.1 src/index.css

```
* { margin: 0; padding: 0; box-sizing: border-box; }

body {
  font-family: "Inter", "Segoe UI", sans-serif;
  background: #f5f3ff;
  color: #1e1b4b;
}

:root {
  --purple-50:  #f5f3ff;
  --purple-100: #ede9fe;
  --purple-200: #ddd6fe;
  --purple-500: #8b5cf6;
  --purple-600: #7c3aed;
  --purple-700: #6d28d9;
  --purple-800: #5b21b6;
  --purple-900: #4c1d95;
  --white: #ffffff;
  --gray-100: #f3f4f6;
  --gray-200: #e5e7eb;
  --gray-600: #4b5563;
  --shadow: 0 4px 6px -1px rgba(124,58,237,0.1), 0 2px 4px -1px rgba(124,58,237,0.06);
  --shadow-lg: 0 10px 15px -3px rgba(124,58,237,0.15);
}

/* LAYOUT */
.layout { display: flex; min-height: 100vh; }
.main-content { flex: 1; margin-left: 260px; padding: 24px; }

/* SIDEBAR */
.sidebar {
  width: 260px; min-height: 100vh;
  background: linear-gradient(180deg, #4c1d95 0%, #7c3aed 100%);
  position: fixed; left: 0; top: 0;
  display: flex; flex-direction: column;
  padding: 0; box-shadow: var(--shadow-lg);
  z-index: 100;
}
.sidebar-logo {
  padding: 28px 24px 24px;
  border-bottom: 1px solid rgba(255,255,255,0.15);
}
.sidebar-logo h1 {
  color: white; font-size: 26px; font-weight: 800;
  letter-spacing: -0.5px;
}
.sidebar-logo span { color: #c4b5fd; font-size: 13px; }
.sidebar-nav { padding: 16px 0; flex: 1; }
.nav-item {
  display: flex; align-items: center; gap: 12px;
  padding: 12px 24px; color: rgba(255,255,255,0.75);
  text-decoration: none; font-size: 14px; font-weight: 500;
  transition: all 0.2s; cursor: pointer; border: none;
  background: none; width: 100%; text-align: left;
}
.nav-item:hover, .nav-item.active {
  background: rgba(255,255,255,0.15);
  color: white; border-radius: 0 24px 24px 0;
```

```
  margin-right: 16px;
}
.nav-item svg { width: 18px; height: 18px; }

/* NAVBAR */
.navbar {
  background: white; padding: 16px 24px;
  display: flex; justify-content: space-between; align-items: center;
  border-radius: 16px; margin-bottom: 24px;
  box-shadow: var(--shadow);
}
.navbar h2 { font-size: 22px; font-weight: 700; color: var(--purple-900); }
.user-badge {
  background: var(--purple-100); padding: 8px 16px;
  border-radius: 24px; font-size: 13px; font-weight: 600;
  color: var(--purple-700);
}

/* CARDS */
.card {
  background: white; border-radius: 16px;
  padding: 24px; box-shadow: var(--shadow);
  border: 1px solid var(--purple-100);
}
.stat-card {
  background: linear-gradient(135deg, #7c3aed 0%, #4c1d95 100%);
  border-radius: 16px; padding: 24px; color: white;
  box-shadow: var(--shadow-lg);
}
.stat-card.light {
  background: white; color: var(--purple-900);
  border: 2px solid var(--purple-200);
}
.stat-card h3 { font-size: 14px; opacity: 0.85; margin-bottom: 8px; }
.stat-card .number { font-size: 36px; font-weight: 800; }
.stats-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px,
1fr)); gap: 16px; }

/* TABLE */
.table-wrapper { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; font-size: 14px; }
th {
  background: var(--purple-900); color: white;
  padding: 12px 16px; text-align: left; font-weight: 600;
}
td { padding: 12px 16px; border-bottom: 1px solid var(--gray-200); }
tr:hover td { background: var(--purple-50); }

/* FORMS */
.form-group { margin-bottom: 18px; }
label { display: block; margin-bottom: 6px; font-weight: 500; font-size: 14px; color:
var(--purple-900); }
input, select, textarea {
  width: 100%; padding: 10px 14px;
  border: 2px solid var(--purple-200);
  border-radius: 10px; font-size: 14px; outline: none;
  transition: border-color 0.2s;
}
input:focus, select:focus { border-color: var(--purple-500); }

/* BUTTONS */
.btn {
  padding: 10px 20px; border: none; border-radius: 10px;
  cursor: pointer; font-size: 14px; font-weight: 600;
  transition: all 0.2s; display: inline-flex; align-items: center; gap: 8px;
```

```
}
.btn-primary { background: var(--purple-600); color: white; }
.btn-primary:hover { background: var(--purple-700); transform: translateY(-1px); }
.btn-danger { background: #ef4444; color: white; }
.btn-danger:hover { background: #dc2626; }
.btn-success { background: #10b981; color: white; }
.btn-outline { background: transparent; border: 2px solid var(--purple-600); color:
var(--purple-600); }

/* BADGE */
.badge {
  padding: 4px 12px; border-radius: 24px;
  font-size: 12px; font-weight: 600;
}
.badge-green { background: #d1fae5; color: #065f46; }
.badge-red { background: #fee2e2; color: #991b1b; }
.badge-yellow { background: #fef3c7; color: #92400e; }
.badge-purple { background: var(--purple-100); color: var(--purple-700); }

/* AUTH PAGES */
.auth-container {
  min-height: 100vh;
  background: linear-gradient(135deg, #4c1d95 0%, #7c3aed 50%, #a78bfa 100%);
  display: flex; align-items: center; justify-content: center;
}
.auth-card {
  background: white; border-radius: 24px;
  padding: 40px; width: 420px; box-shadow: var(--shadow-lg);
}
.auth-card h2 { font-size: 28px; font-weight: 800; color: var(--purple-900); margin-
bottom: 8px; }
.auth-card p { color: var(--gray-600); margin-bottom: 28px; font-size: 14px; }

/* RESPONSIVE */
@media (max-width: 768px) {
  .sidebar { transform: translateX(-100%); transition: 0.3s; }
  .sidebar.open { transform: translateX(0); }
  .main-content { margin-left: 0; }
  .stats-grid { grid-template-columns: 1fr 1fr; }
}
@media (max-width: 480px) {
  .stats-grid { grid-template-columns: 1fr; }
  .auth-card { width: 90%; padding: 24px; }
}

/* DARK MODE */
.dark body { background: #1e1b4b; color: #e0e7ff; }
.dark .card, .dark .navbar { background: #2d2b6b; border-color: #4c1d95; }
.dark input, .dark select { background: #1e1b4b; color: #e0e7ff; border-color:
#4c1d95; }
```

SECTION 17: SIDEBAR & NAVBAR COMPONENTS

17.1 src/components/Sidebar.jsx

```
import { Link, useNavigate, useLocation } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import {
  LayoutDashboard, Users, UserCheck, BookOpen,
  CalendarCheck, BarChart2, ClipboardList, LogOut
} from "lucide-react";

const Sidebar = () => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  const handleLogout = () => { logout(); navigate("/login"); };

  const adminLinks = [
    { to: "/admin",     label: "Dashboard",  icon: <LayoutDashboard /> },
    { to: "/students",  label: "Students",   icon: <Users /> },
    { to: "/teachers",  label: "Teachers",   icon: <UserCheck /> },
    { to: "/attendance",label: "Attendance", icon: <CalendarCheck /> },
    { to: "/marks",     label: "Marks",      icon: <ClipboardList /> },
    { to: "/analytics", label: "Analytics",  icon: <BarChart2 /> },
  ];

  const teacherLinks = [
    { to: "/teacher",   label: "Dashboard",  icon: <LayoutDashboard /> },
    { to: "/students",  label: "Students",   icon: <Users /> },
    { to: "/attendance",label: "Attendance", icon: <CalendarCheck /> },
    { to: "/marks",     label: "Marks",      icon: <ClipboardList /> },
    { to: "/analytics", label: "Analytics",  icon: <BarChart2 /> },
  ];

  const studentLinks = [
    { to: "/student",   label: "Dashboard",  icon: <LayoutDashboard /> },
    { to: "/attendance",label: "My Attendance", icon: <CalendarCheck /> },
    { to: "/marks",     label: "My Marks",   icon: <ClipboardList /> },
    { to: "/analytics", label: "Analytics",  icon: <BarChart2 /> },
  ];

  const links = user?.role === "ADMIN" ? adminLinks
              : user?.role === "TEACHER" ? teacherLinks : studentLinks;

  return (
    <div className="sidebar">
      <div className="sidebar-logo">
        <h1>🎓 EduFlow</h1>
        <span>{user?.role}</span>
      </div>
      <nav className="sidebar-nav">
        {links.map(link => (
          <Link
            key={link.to}
            to={link.to}
            className={`nav-item ${location.pathname === link.to ? "active" : ""}`}
          >
            {link.icon} {link.label}
          </Link>
        ))}
```

```
      </nav>
      <button className="nav-item" onClick={handleLogout}
style={{marginBottom:"16px"}}>
        <LogOut size={18} /> Logout
      </button>
    </div>
  );
};

export default Sidebar;
```

17.2 src/components/Navbar.jsx

```
import { useAuth } from "../context/AuthContext";

const Navbar = ({ title }) => {
  const { user } = useAuth();
  return (
    <div className="navbar">
      <h2>{title}</h2>
      <div className="user-badge">
        👤 {user?.name} • {user?.role}
      </div>
    </div>
  );
};

export default Navbar;
```

17.3 src/components/StatCard.jsx

```
const StatCard = ({ title, value, icon, light }) => (
  <div className={`stat-card ${light ? "light" : ""}`}>
    <div style={{ display:"flex", justifyContent:"space-between",
alignItems:"center" }}>
      <div>
        <h3>{title}</h3>
        <div className="number">{value}</div>
      </div>
      <div style={{ fontSize: "36px" }}>{icon}</div>
    </div>
  </div>
);

export default StatCard;
```

SECTION 18: AUTH PAGES (LOGIN, SIGNUP, OTP, FORGOT)

18.1 src/pages/Login.jsx

```
import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import API from "../api/axiosConfig";

const Login = () => {
  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true); setError("");
    try {
      const res = await API.post("/api/auth/login", form);
      login(res.data);
      if (res.data.role === "ADMIN") navigate("/admin");
      else if (res.data.role === "TEACHER") navigate("/teacher");
      else navigate("/student");
    } catch (err) {
      setError(err.response?.data?.error || "Login failed. Check credentials.");
    } finally { setLoading(false); }
  };

  return (
    <div className="auth-container">
      <div className="auth-card">
        <h2>🎓 EduFlow</h2>
        <p>Sign in to your account</p>
        {error && <div style={{color:"red",marginBottom:"16px",fontSize:"14px"}}
>{error}</div>}
        <form onSubmit={handleSubmit}>
          <div className="form-group">
            <label>Email Address</label>
            <input type="email" placeholder="Enter your email"
              value={form.email}
              onChange={e => setForm({...form, email: e.target.value})} required />
          </div>
          <div className="form-group">
            <label>Password</label>
            <input type="password" placeholder="Enter your password"
              value={form.password}
              onChange={e => setForm({...form, password: e.target.value})} required />
          </div>
          <button className="btn btn-primary"
style={{width:"100%",justifyContent:"center"}}
            type="submit" disabled={loading}>
            {loading ? "Signing in..." : "Sign In"}
          </button>
        </form>
        <div style={{marginTop:"20px",textAlign:"center",fontSize:"14px"}}>
          <Link to="/forgot-password" style={{color:"#7c3aed"}}>Forgot
Password?</Link>
          <span style={{margin:"0 8px"}}>•</span>
          <Link to="/signup" style={{color:"#7c3aed"}}>Create Account</Link>
        </div>
```

```
      </div>
    </div>
  );
};

export default Login;
```

18.2 src/pages/Signup.jsx

```
import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import API from "../api/axiosConfig";

const Signup = () => {
  const [step, setStep] = useState(1);
  const [form, setForm] = useState({
    name: "", email: "", password: "", role: "STUDENT"
  });
  const [otp, setOtp] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  // Step 1: Send OTP
  const sendOtp = async (e) => {
    e.preventDefault();
    setLoading(true); setError("");
    try {
      await API.post(`/api/auth/send-otp?email=${form.email}`);
      setStep(2);
    } catch (err) {
      setError(err.response?.data?.error || "Failed to send OTP.");
    } finally { setLoading(false); }
  };

  // Step 2: Register with OTP
  const register = async (e) => {
    e.preventDefault();
    setLoading(true); setError("");
    try {
      await API.post(`/api/auth/register?otp=${otp}`, form);
      navigate("/login");
    } catch (err) {
      setError(err.response?.data?.error || "Registration failed.");
    } finally { setLoading(false); }
  };

  return (
    <div className="auth-container">
      <div className="auth-card">
        <h2>Create Account</h2>
        <p>Join EduFlow today</p>
        {error && <div style={{color:"red",marginBottom:"16px",fontSize:"14px"}}
>{error}</div>}

        {step === 1 ? (
          <form onSubmit={sendOtp}>
            <div className="form-group">
              <label>Full Name</label>
              <input type="text" placeholder="Your full name"
                value={form.name}
                onChange={e => setForm({...form, name: e.target.value})} required />
            </div>
```

```
            <div className="form-group">
              <label>Email</label>
              <input type="email" placeholder="Your email"
                value={form.email}
                onChange={e => setForm({...form, email: e.target.value})} required />
            </div>
            <div className="form-group">
              <label>Password</label>
              <input type="password" placeholder="Min 6 characters"
                value={form.password}
                onChange={e => setForm({...form, password: e.target.value})}
required />
            </div>
            <div className="form-group">
              <label>Role</label>
              <select value={form.role}
                onChange={e => setForm({...form, role: e.target.value})}>
                <option value="STUDENT">Student</option>
                <option value="TEACHER">Teacher</option>
                <option value="ADMIN">Admin</option>
              </select>
            </div>
            <button className="btn btn-primary"
style={{width:"100%",justifyContent:"center"}}
              type="submit" disabled={loading}>
              {loading ? "Sending OTP..." : "Send OTP"}
            </button>
          </form>
        ) : (
          <form onSubmit={register}>
            <p style={{marginBottom:"16px",color:"#6b7280"}}>
              OTP sent to {form.email}</p>
            <div className="form-group">
              <label>Enter OTP</label>
              <input type="text" placeholder="6-digit OTP"
                value={otp} onChange={e => setOtp(e.target.value)
                } maxLength={6} required />
            </div>
            <button className="btn btn-primary"
style={{width:"100%",justifyContent:"center"}}
              type="submit" disabled={loading}>
              {loading ? "Verifying..." : "Complete Registration"}
            </button>
            <button type="button" className="btn btn-outline"
              style={{width:"100%",justifyContent:"center",marginTop:"10px"}}
              onClick={() => setStep(1)}>
              Back
            </button>
          </form>
        )}
        <div style={{marginTop:"20px",textAlign:"center",fontSize:"14px"}}>
          Already have an account? <Link to="/login" style={{color:"#7c3aed"}}>Sign
In</Link>
        </div>
      </div>
    </div>
  );
};

export default Signup;
```

18.3 src/pages/ForgotPassword.jsx

```
import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
```

```
import API from "../api/axiosConfig";

const ForgotPassword = () => {
  const [step, setStep] = useState(1);
  const [email, setEmail] = useState("");
  const [otp, setOtp] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const sendOtp = async (e) => {
    e.preventDefault(); setLoading(true); setError("");
    try {
      await API.post(`/api/auth/forgot-password?email=${email}`);
      setStep(2);
    } catch (err) { setError(err.response?.data?.error || "Error"); }
    finally { setLoading(false); }
  };

  const resetPassword = async (e) => {
    e.preventDefault(); setLoading(true); setError("");
    try {
      await API.post("/api/auth/reset-password", { email, otp, newPassword });
      navigate("/login");
    } catch (err) { setError(err.response?.data?.error || "Error"); }
    finally { setLoading(false); }
  };

  return (
    <div className="auth-container">
      <div className="auth-card">
        <h2>Reset Password</h2>
        <p>We will send an OTP to your email</p>
        {error && <div style={{color:"red",marginBottom:"16px",fontSize:"14px"}}
>{error}</div>}
        {step === 1 ? (
          <form onSubmit={sendOtp}>
            <div className="form-group">
              <label>Email Address</label>
              <input type="email" placeholder="Your registered email"
                value={email} onChange={e => setEmail(e.target.value)} required />
            </div>
            <button className="btn btn-primary"
style={{width:"100%",justifyContent:"center"}}
              type="submit" disabled={loading}>
              {loading ? "Sending..." : "Send OTP"}
            </button>
          </form>
        ) : (
          <form onSubmit={resetPassword}>
            <div className="form-group">
              <label>OTP</label>
              <input type="text" placeholder="6-digit OTP"
                value={otp} onChange={e => setOtp(e.target.value)} required />
            </div>
            <div className="form-group">
              <label>New Password</label>
              <input type="password" placeholder="Min 6 characters"
                value={newPassword}
                onChange={e => setNewPassword(e.target.value)} required />
            </div>
            <button className="btn btn-primary"
style={{width:"100%",justifyContent:"center"}}
              type="submit" disabled={loading}>
```

```
              {loading ? "Resetting..." : "Reset Password"}
            </button>
          </form>
        )}
        <div style={{marginTop:"16px",textAlign:"center",fontSize:"14px"}}>
          <Link to="/login" style={{color:"#7c3aed"}}>Back to Login</Link>
        </div>
      </div>
    </div>
  );
};

export default ForgotPassword;
```

SECTION 19: DASHBOARD PAGES

19.1 src/pages/AdminDashboard.jsx

```
import { useState, useEffect } from "react";
import Sidebar from "../components/Sidebar";
import Navbar from "../components/Navbar";
import StatCard from "../components/StatCard";
import Spinner from "../components/Spinner";
import API from "../api/axiosConfig";

const AdminDashboard = () => {
  const [stats, setStats] = useState(null);
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const [statsRes, studentsRes] = await Promise.all([
          API.get("/api/analytics/dashboard"),
          API.get("/api/students"),
        ]);
        setStats(statsRes.data);
        setStudents(studentsRes.data.slice(0, 5)); // recent 5
      } catch (err) { console.error(err); }
      finally { setLoading(false); }
    };
    fetchData();
  }, []);

  if (loading) return (
    <div className="layout"><Sidebar /><div className="main-content"><Spinner
/></div></div>
  );

  return (
    <div className="layout">
      <Sidebar />
      <div className="main-content">
        <Navbar title="Admin Dashboard" />

        <div className="stats-grid" style={{marginBottom:"24px"}}>
          <StatCard title="Total Students" value={stats?.totalStudents || 0} icon="👨‍🎓"
/>
          <StatCard title="Attendance Records" value={stats?.totalAttendanceRecords ||
0} icon="📅" />
          <StatCard title="Marks Records" value={stats?.totalMarksRecords || 0}
icon="📊" />
          <StatCard title="Status" value="Active" icon="✅" light />
        </div>

        <div className="card">
          <h3
style={{marginBottom:"16px",fontSize:"18px",fontWeight:"700",color:"#4c1d95"}}>
            Recent Students
          </h3>
          <div className="table-wrapper">
            <table>
              <thead>
                <tr>
```

```
                <th>Name</th><th>Email</th><th>Roll No</th><th>Course</th>
              </tr>
            </thead>
            <tbody>
              {students.map(s => (
                <tr key={s.id}>
                  <td>{s.name}</td>
                  <td>{s.email}</td>
                  <td>{s.rollNumber || "-"}</td>
                  <td>{s.courseName || "-"}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>
  );
};

export default AdminDashboard;
```

19.2 src/pages/TeacherDashboard.jsx

```
import { useState, useEffect } from "react";
import Sidebar from "../components/Sidebar";
import Navbar from "../components/Navbar";
import StatCard from "../components/StatCard";
import Spinner from "../components/Spinner";
import API from "../api/axiosConfig";

const TeacherDashboard = () => {
  const [students, setStudents] = useState([]);
  const [attendance, setAttendance] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const [studentsRes, attRes] = await Promise.all([
          API.get("/api/students"),
          API.get("/api/attendance"),
        ]);
        setStudents(studentsRes.data);
        setAttendance(attRes.data);
      } catch (err) { console.error(err); }
      finally { setLoading(false); }
    };
    fetchData();
  }, []);

  if (loading) return (
    <div className="layout"><Sidebar /><div className="main-content"><Spinner
/></div></div>
  );

  const presentCount = attendance.filter(a => a.status === "PRESENT").length;
  const attPct = attendance.length === 0 ? 0 :
    Math.round((presentCount / attendance.length) * 100);

  return (
    <div className="layout">
```

```
      <Sidebar />
      <div className="main-content">
        <Navbar title="Teacher Dashboard" />
        <div className="stats-grid" style={{marginBottom:"24px"}}>
          <StatCard title="My Students" value={students.length} icon="👨‍🎓" />
          <StatCard title="Attendance Records" value={attendance.length} icon="📅" />
          <StatCard title="Avg Attendance %" value={`${attPct}%`} icon="📈" />
          <StatCard title="Active" value="Today" icon="🟢" light />
        </div>
        <div className="card">
          <h3
style={{marginBottom:"16px",fontSize:"18px",fontWeight:"700",color:"#4c1d95"}}>
            Quick Actions
          </h3>
          <div style={{display:"flex",gap:"12px",flexWrap:"wrap"}}>
            <a href="/students" className="btn btn-primary">➕ Add Student</a>
            <a href="/attendance" className="btn btn-success">📅 Mark Attendance</a>
            <a href="/marks" className="btn btn-outline">📊 Add Marks</a>
          </div>
        </div>
      </div>
    </div>
  );
};

export default TeacherDashboard;
```

19.3 src/pages/StudentDashboard.jsx

```
import { useState, useEffect } from "react";
import { useAuth } from "../context/AuthContext";
import Sidebar from "../components/Sidebar";
import Navbar from "../components/Navbar";
import StatCard from "../components/StatCard";
import Spinner from "../components/Spinner";
import API from "../api/axiosConfig";

const StudentDashboard = () => {
  const { user } = useAuth();
  const [analytics, setAnalytics] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        // Get all students to find this student's ID
        const studentsRes = await API.get("/api/students");
        const myStudent = studentsRes.data.find(s => s.email === user.email);
        if (myStudent) {
          const res = await API.get(`/api/analytics/student/${myStudent.id}`);
          setAnalytics(res.data);
        }
      } catch (err) { console.error(err); }
      finally { setLoading(false); }
    };
    fetchData();
  }, [user]);

  if (loading) return (
    <div className="layout"><Sidebar /><div className="main-content"><Spinner
/></div></div>
  );
```

```
  return (
    <div className="layout">
      <Sidebar />
      <div className="main-content">
        <Navbar title={`Welcome, ${user?.name}!`} />
        <div className="stats-grid" style={{marginBottom:"24px"}}>
          <StatCard title="Attendance %" value={`${analytics?.attendancePercentage ||
0}%`} icon="📅" />
          <StatCard title="Average Marks" value={`${analytics?.averageMarks || 0}%`}
icon="📊" />
          <StatCard title="Performance" value={analytics?.performanceStatus || "-"}
icon="🏆" />
          <StatCard title="Subjects" value={Object.keys(analytics?.subjectMarks ||
{}).length} icon="📚" light />
        </div>

        {analytics && (
          <div className="card">
            <h3
style={{marginBottom:"16px",fontSize:"18px",fontWeight:"700",color:"#4c1d95"}}>
              Subject Performance
            </h3>
            <table>
              <thead><tr><th>Subject</th><th>Marks Obtained</th></tr></thead>
              <tbody>
                {Object.entries(analytics.subjectMarks).map(([sub, marks]) => (
                  <tr key={sub}>
                    <td>{sub}</td>
                    <td><span className="badge badge-purple">{marks}</span></td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}

        <div className="card" style={{marginTop:"16px"}}>
          <h3
style={{marginBottom:"16px",fontSize:"18px",fontWeight:"700",color:"#4c1d95"}}>
            Download Reports
          </h3>
          <div style={{display:"flex",gap:"12px"}}>
            <a href={`${import.meta.env.VITE_API_URL}/api/pdf/marksheet/$
{analytics?.studentId}`}
               className="btn btn-primary" target="_blank" rel="noreferrer">
               📄 Download Marksheet
            </a>
            <a href={`${import.meta.env.VITE_API_URL}/api/pdf/attendance/$
{analytics?.studentId}`}
               className="btn btn-outline" target="_blank" rel="noreferrer">
               📅 Download Attendance
            </a>
          </div>
        </div>
      </div>
    </div>
  );
};

export default StudentDashboard;
```

SECTION 20: STUDENTS, ATTENDANCE & MARKS PAGES

20.1 src/pages/Students.jsx

```
import { useState, useEffect } from "react";
import Sidebar from "../components/Sidebar";
import Navbar from "../components/Navbar";
import Spinner from "../components/Spinner";
import API from "../api/axiosConfig";

const Students = () => {
  const [students, setStudents] = useState([]);
  const [search, setSearch] = useState("");
  const [loading, setLoading] = useState(true);
  const [showForm, setShowForm] = useState(false);
  const [form, setForm] = useState({
    name:"",email:"",rollNumber:"",phone:"",gender:"",courseId:""
  });

  const fetchStudents = async () => {
    setLoading(true);
    try {
      const res = await API.get("/api/students");
      setStudents(res.data);
    } catch (err) { console.error(err); }
    finally { setLoading(false); }
  };

  useEffect(() => { fetchStudents(); }, []);

  const handleSearch = async (e) => {
    const val = e.target.value;
    setSearch(val);
    if (val.trim()) {
      const res = await API.get(`/api/students/search?name=${val}`);
      setStudents(res.data);
    } else { fetchStudents(); }
  };

  const handleAdd = async (e) => {
    e.preventDefault();
    try {
      await API.post("/api/students", form);
      setShowForm(false);
      setForm({name:"",email:"",rollNumber:"",phone:"",gender:"",courseId:""});
      fetchStudents();
    } catch (err) { alert(err.response?.data?.error || "Error adding student"); }
  };

  const handleDelete = async (id) => {
    if (!window.confirm("Delete this student?")) return;
    await API.delete(`/api/students/${id}`);
    fetchStudents();
  };

  return (
    <div className="layout">
      <Sidebar />
      <div className="main-content">
        <Navbar title="Students Management" />
```

```
        <div style={{display:"flex",gap:"12px",marginBottom:"20px",flexWrap:"wrap"}}>
          <input placeholder="🔍 Search by name..." value={search}
            onChange={handleSearch}
            style={{flex:1,minWidth:"200px",padding:"10px 14px",
              borderRadius:"10px",border:"2px solid #ddd6fe"}} />
          <button className="btn btn-primary" onClick={() => setShowForm(!showForm)}>
            ➕ Add Student
          </button>
        </div>

        {showForm && (
          <div className="card" style={{marginBottom:"20px"}}>
            <h3 style={{marginBottom:"16px",fontWeight:"700",color:"#4c1d95"}}>Add New
Student</h3>
            <form onSubmit={handleAdd}
              style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:"12px"}}>
              <div className="form-group">
                <label>Full Name</label>
                <input required value={form.name}
                  onChange={e=>setForm({...form,name:e.target.value})} />
              </div>
              <div className="form-group">
                <label>Email</label>
                <input type="email" required value={form.email}
                  onChange={e=>setForm({...form,email:e.target.value})} />
              </div>
              <div className="form-group">
                <label>Roll Number</label>
                <input value={form.rollNumber}
                  onChange={e=>setForm({...form,rollNumber:e.target.value})} />
              </div>
              <div className="form-group">
                <label>Phone</label>
                <input value={form.phone}
                  onChange={e=>setForm({...form,phone:e.target.value})} />
              </div>
              <div className="form-group">
                <label>Gender</label>
                <select value={form.gender}
                  onChange={e=>setForm({...form,gender:e.target.value})}>
                  <option value="">Select</option>
                  <option value="Male">Male</option>
                  <option value="Female">Female</option>
                  <option value="Other">Other</option>
                </select>
              </div>
              <div style={{gridColumn:"1/-1",display:"flex",gap:"10px"}}>
                <button type="submit" className="btn btn-primary">Save
Student</button>
                <button type="button" className="btn btn-outline"
                  onClick={() => setShowForm(false)}>Cancel</button>
              </div>
            </form>
          </div>
        )}

        <div className="card">
          {loading ? <Spinner /> : (
            <div className="table-wrapper">
              <table>
                <thead><tr>
                  <th>Name</th><th>Email</th><th>Roll No</th>
                  <th>Course</th><th>Actions</th>
                </tr></thead>
                <tbody>
                  {students.map(s => (
```

```
                    <tr key={s.id}>
                      <td>{s.name}</td>
                      <td>{s.email}</td>
                      <td>{s.rollNumber || "-"}</td>
                      <td>{s.courseName || "-"}</td>
                      <td style={{display:"flex",gap:"8px"}}>
                        <a href={`${import.meta.env.VITE_API_URL}/api/pdf/marksheet/$
{s.id}`}
                           target="_blank" rel="noreferrer"
                           className="btn btn-outline" style={{padding:"6px
12px",fontSize:"12px"}}>
                           📄
                        </a>
                        <button className="btn btn-danger"
                          style={{padding:"6px 12px",fontSize:"12px"}}
                          onClick={() => handleDelete(s.id)}>
                          🗑️
                        </button>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
              {students.length === 0 && (
                <p style={{textAlign:"center",padding:"32px",color:"#9ca3af"}}>
                  No students found.
                </p>
              )}
            </div>
          )}
        </div>
      </div>
    </div>
  );
};

export default Students;
```

20.2 src/pages/Attendance.jsx

```
import { useState, useEffect } from "react";
import { useAuth } from "../context/AuthContext";
import Sidebar from "../components/Sidebar";
import Navbar from "../components/Navbar";
import Spinner from "../components/Spinner";
import API from "../api/axiosConfig";

const Attendance = () => {
  const { user } = useAuth();
  const [attendance, setAttendance] = useState([]);
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [form, setForm] = useState({
    studentId: "", date: new Date().toISOString().split("T")[0],
    status: "PRESENT", subject: ""
  });

  const fetchData = async () => {
    setLoading(true);
    try {
      const [studRes, attRes] = await Promise.all([
        API.get("/api/students"),
        API.get("/api/attendance"),
      ]);
```

```
      setStudents(studRes.data);
      if (user.role === "STUDENT") {
        const me = studRes.data.find(s => s.email === user.email);
        if (me) {
          const res = await API.get(`/api/attendance/student/${me.id}`);
          setAttendance(res.data);
        }
      } else {
        setAttendance(attRes.data);
      }
    } catch(e) { console.error(e); }
    finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, []);

  const handleMark = async (e) => {
    e.preventDefault();
    try {
      await API.post("/api/attendance", form);
      fetchData();
      setForm({...form, studentId: "", subject: ""});
    } catch(err) { alert(err.response?.data?.error || "Error"); }
  };

  return (
    <div className="layout">
      <Sidebar />
      <div className="main-content">
        <Navbar title="Attendance" />

        {(user.role === "TEACHER" || user.role === "ADMIN") && (
          <div className="card" style={{marginBottom:"20px"}}>
            <h3 style={{marginBottom:"16px",fontWeight:"700",color:"#4c1d95"}}>Mark
Attendance</h3>
            <form onSubmit={handleMark}
              style={{display:"grid",gridTemplateColumns:"repeat(auto-
fit,minmax(180px,1fr))",gap:"12px"}}>
              <div className="form-group">
                <label>Student</label>
                <select required value={form.studentId}
                  onChange={e=>setForm({...form,studentId:e.target.value})}>
                  <option value="">Select Student</option>
                  {students.map(s => (
                    <option key={s.id} value={s.id}>{s.name}</option>
                  ))}
                </select>
              </div>
              <div className="form-group">
                <label>Subject</label>
                <input required value={form.subject}
                  onChange={e=>setForm({...form,subject:e.target.value})} />
              </div>
              <div className="form-group">
                <label>Date</label>
                <input type="date" required value={form.date}
                  onChange={e=>setForm({...form,date:e.target.value})} />
              </div>
              <div className="form-group">
                <label>Status</label>
                <select value={form.status}
                  onChange={e=>setForm({...form,status:e.target.value})}>
                  <option value="PRESENT">Present</option>
                  <option value="ABSENT">Absent</option>
                </select>
```

```
              </div>
              <div className="form-group" style={{display:"flex",alignItems:"flex-
end"}}>
                <button className="btn btn-primary" type="submit">Mark</button>
              </div>
            </form>
          </div>
        )}

        <div className="card">
          {loading ? <Spinner /> : (
            <div className="table-wrapper">
              <table>
                <thead><tr>
                  <th>Student</th><th>Subject</th><th>Date</th><th>Status</th>
                </tr></thead>
                <tbody>
                  {attendance.map(a => (
                    <tr key={a.id}>
                      <td>{a.studentName}</td>
                      <td>{a.subject}</td>
                      <td>{a.date}</td>
                      <td>
                        <span className={`badge ${a.status===="PRESENT" ? "badge-green"
: "badge-red"}`}>
                          {a.status}
                        </span>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          )}
        </div>
      </div>
    </div>
  );
};

export default Attendance;
```

20.3 src/pages/Marks.jsx

```
import { useState, useEffect } from "react";
import { useAuth } from "../context/AuthContext";
import Sidebar from "../components/Sidebar";
import Navbar from "../components/Navbar";
import Spinner from "../components/Spinner";
import API from "../api/axiosConfig";

const Marks = () => {
  const { user } = useAuth();
  const [marks, setMarks] = useState([]);
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [form, setForm] = useState({

studentId:"",subject:"",marksObtained:0,totalMarks:100,examType:"MID_TERM",semester:"1
"
  });

  const fetchData = async () => {
    setLoading(true);
```

```
    try {
      const studRes = await API.get("/api/students");
      setStudents(studRes.data);
      if (user.role === "STUDENT") {
        const me = studRes.data.find(s => s.email === user.email);
        if (me) {
          const res = await API.get(`/api/marks/student/${me.id}`);
          setMarks(res.data);
        }
      } else {
        const res = await API.get("/api/marks");
        setMarks(res.data);
      }
    } catch(e) { console.error(e); }
    finally { setLoading(false); }
  };

  useEffect(() => { fetchData(); }, []);

  const handleAdd = async (e) => {
    e.preventDefault();
    try {
      await API.post("/api/marks", form);
      fetchData();

setForm({studentId:"",subject:"",marksObtained:0,totalMarks:100,examType:"MID_TERM",se
mester:"1"});
    } catch(err) { alert(err.response?.data?.error || "Error"); }
  };

  return (
    <div className="layout">
      <Sidebar />
      <div className="main-content">
        <Navbar title="Marks Management" />

        {(user.role === "TEACHER" || user.role === "ADMIN") && (
          <div className="card" style={{marginBottom:"20px"}}>
            <h3 style={{marginBottom:"16px",fontWeight:"700",color:"#4c1d95"}}>Add
Marks</h3>
            <form onSubmit={handleAdd}
              style={{display:"grid",gridTemplateColumns:"repeat(auto-
fit,minmax(160px,1fr))",gap:"12px"}}>
              <div className="form-group">
                <label>Student</label>
                <select required value={form.studentId}
                  onChange={e=>setForm({...form,studentId:e.target.value})}>
                  <option value="">Select</option>
                  {students.map(s=>(<option key={s.id}
value={s.id}>{s.name}</option>))}
                </select>
              </div>
              <div className="form-group">
                <label>Subject</label>
                <input required value={form.subject}
                  onChange={e=>setForm({...form,subject:e.target.value})} />
              </div>
              <div className="form-group">
                <label>Marks Obtained</label>
                <input type="number" required value={form.marksObtained}

onChange={e=>setForm({...form,marksObtained:parseInt(e.target.value)})} />
              </div>
              <div className="form-group">
                <label>Total Marks</label>
```

```
                <input type="number" required value={form.totalMarks}
                  onChange={e=>setForm({...form,totalMarks:parseInt(e.target.value)})}
/>
              </div>
              <div className="form-group">
                <label>Exam Type</label>
                <select value={form.examType}
                  onChange={e=>setForm({...form,examType:e.target.value})}>
                  <option value="MID_TERM">Mid Term</option>
                  <option value="FINAL">Final</option>
                  <option value="QUIZ">Quiz</option>
                </select>
              </div>
              <div className="form-group">
                <label>Semester</label>
                <input value={form.semester}
                  onChange={e=>setForm({...form,semester:e.target.value})} />
              </div>
              <div className="form-group" style={{display:"flex",alignItems:"flex-
end"}}>
                <button className="btn btn-primary" type="submit">Add Marks</button>
              </div>
            </form>
          </div>
        )}

        <div className="card">
          {loading ? <Spinner /> : (
            <div className="table-wrapper">
              <table>
                <thead><tr>
                  <th>Student</th><th>Subject</th>
                  <th>Marks</th><th>Total</th><th>%</th><th>Exam Type</th>
                </tr></thead>
                <tbody>
                  {marks.map(m => {
                    const pct = Math.round((m.marksObtained/m.totalMarks)*100);
                    return (
                      <tr key={m.id}>
                        <td>{m.studentName}</td>
                        <td>{m.subject}</td>
                        <td>{m.marksObtained}</td>
                        <td>{m.totalMarks}</td>
                        <td>
                          <span className={`badge ${pct>=75?"badge-
green":pct>=50?"badge-yellow":"badge-red"}`}>
                            {pct}%
                          </span>
                        </td>
                        <td>{m.examType}</td>
                      </tr>
                    );
                  })}
                </tbody>
              </table>
            </div>
          )}
        </div>
      </div>
    </div>
  );
};

export default Marks;
```

SECTION 21: ANALYTICS PAGE WITH CHARTS

21.1 src/pages/Analytics.jsx

```
import { useState, useEffect } from "react";
import { useAuth } from "../context/AuthContext";
import Sidebar from "../components/Sidebar";
import Navbar from "../components/Navbar";
import Spinner from "../components/Spinner";
import API from "../api/axiosConfig";
import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip,
  PieChart, Pie, Cell, Legend, ResponsiveContainer
} from "recharts";

const COLORS = ["#7c3aed", "#ef4444", "#10b981", "#f59e0b", "#3b82f6"];

const Analytics = () => {
  const { user } = useAuth();
  const [students, setStudents] = useState([]);
  const [selectedId, setSelectedId] = useState("");
  const [analytics, setAnalytics] = useState(null);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    const fetchStudents = async () => {
      const res = await API.get("/api/students");
      setStudents(res.data);
      // Auto-select for student role
      if (user.role === "STUDENT") {
        const me = res.data.find(s => s.email === user.email);
        if (me) { setSelectedId(me.id); fetchAnalytics(me.id); }
      }
    };
    fetchStudents();
  }, [user]);

  const fetchAnalytics = async (id) => {
    setLoading(true);
    try {
      const res = await API.get(`/api/analytics/student/${id}`);
      setAnalytics(res.data);
    } catch(e) { console.error(e); }
    finally { setLoading(false); }
  };

  const handleStudentChange = (e) => {
    setSelectedId(e.target.value);
    if (e.target.value) fetchAnalytics(e.target.value);
    else setAnalytics(null);
  };

  // Prepare chart data
  const barData = analytics ?
    Object.entries(analytics.subjectMarks).map(([sub, marks]) => ({
      subject: sub, marks
    })) : [];

  const pieData = analytics ? [
    { name: "Present", value: analytics.totalPresent },
    { name: "Absent", value: analytics.totalAbsent },
```

```
  ] : [];

  return (
    <div className="layout">
      <Sidebar />
      <div className="main-content">
        <Navbar title="Analytics & Reports" />

        {user.role !== "STUDENT" && (
          <div className="card" style={{marginBottom:"20px"}}>
            <div className="form-group">
              <label>Select Student</label>
              <select value={selectedId} onChange={handleStudentChange}>
                <option value="">-- Choose Student --</option>
                {students.map(s=>(<option key={s.id} value={s.id}>{s.name}</option>))}
              </select>
            </div>
          </div>
        )}

        {loading && <Spinner />}

        {analytics && !loading && (
          <>
            {/* Summary Cards */}
            <div className="stats-grid" style={{marginBottom:"24px"}}>
              <div className="stat-card">
                <h3>Student</h3>
                <div style={{fontSize:"20px",fontWeight:"700"}}
>{analytics.studentName}</div>
              </div>
              <div className="stat-card">
                <h3>Attendance</h3>
                <div className="number">{analytics.attendancePercentage}%</div>
              </div>
              <div className="stat-card">
                <h3>Avg Marks</h3>
                <div className="number">{analytics.averageMarks}%</div>
              </div>
              <div className="stat-card light">
                <h3>Performance</h3>
                <div className="number">{analytics.performanceStatus}</div>
              </div>
            </div>

            {/* Charts */}
            <div style={{display:"grid",gridTemplateColumns:"1fr
1fr",gap:"20px",marginBottom:"20px"}}>
              {/* Bar Chart - Subject Marks */}
              <div className="card">
                <h3 style={{marginBottom:"16px",fontWeight:"700",color:"#4c1d95"}}>
                  Subject-wise Marks
                </h3>
                <ResponsiveContainer width="100%" height={250}>
                  <BarChart data={barData}>
                    <CartesianGrid strokeDasharray="3 3" />
                    <XAxis dataKey="subject" />
                    <YAxis />
                    <Tooltip />
                    <Bar dataKey="marks" fill="#7c3aed" radius={[6,6,0,0]} />
                  </BarChart>
                </ResponsiveContainer>
              </div>

              {/* Pie Chart - Attendance */}
```

```
              <div className="card">
                <h3 style={{marginBottom:"16px",fontWeight:"700",color:"#4c1d95"}}>
                  Attendance Overview
                </h3>
                <ResponsiveContainer width="100%" height={250}>
                  <PieChart>
                    <Pie data={pieData} dataKey="value" nameKey="name"
                      cx="50%" cy="50%" outerRadius={90} label>
                      {pieData.map((_, i) => (
                        <Cell key={i} fill={COLORS[i % COLORS.length]} />
                      ))}
                    </Pie>
                    <Tooltip />
                    <Legend />
                  </PieChart>
                </ResponsiveContainer>
              </div>
            </div>

            {/* Download Buttons */}
            <div className="card">
              <h3 style={{marginBottom:"16px",fontWeight:"700",color:"#4c1d95"}}
>Download Reports</h3>
              <div style={{display:"flex",gap:"12px",flexWrap:"wrap"}}>
                <a href={`${import.meta.env.VITE_API_URL}/api/pdf/marksheet/$
{analytics.studentId}`}
                   target="_blank" rel="noreferrer" className="btn btn-primary">
                   📄 Download Marksheet PDF
                </a>
                <a href={`${import.meta.env.VITE_API_URL}/api/pdf/attendance/$
{analytics.studentId}`}
                   target="_blank" rel="noreferrer" className="btn btn-outline">
                   📅 Download Attendance PDF
                </a>
              </div>
            </div>
          </>
        )}

        {!analytics && !loading && (
          <div className="card" style={{textAlign:"center",padding:"60px"}}>
            <p style={{fontSize:"48px"}}>📊</p>
            <p style={{color:"#9ca3af",marginTop:"12px"}}>
              Select a student to view analytics
            </p>
          </div>
        )}
      </div>
    </div>
  );
};

export default Analytics;
```

SECTION 22: TEACHERS PAGE

22.1 src/pages/Teachers.jsx

```
import { useState, useEffect } from "react";
import Sidebar from "../components/Sidebar";
import Navbar from "../components/Navbar";
import Spinner from "../components/Spinner";
import API from "../api/axiosConfig";

const Teachers = () => {
  const [teachers, setTeachers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showForm, setShowForm] = useState(false);
  const [form, setForm] = useState({
    name:"",email:"",subject:"",phone:"",qualification:""
  });

  const fetchTeachers = async () => {
    setLoading(true);
    try {
      const res = await API.get("/api/teachers");
      setTeachers(res.data);
    } catch(e){ console.error(e); }
    finally { setLoading(false); }
  };

  useEffect(() => { fetchTeachers(); }, []);

  const handleAdd = async (e) => {
    e.preventDefault();
    try {
      await API.post("/api/teachers", form);
      setShowForm(false);
      setForm({name:"",email:"",subject:"",phone:"",qualification:""});
      fetchTeachers();
    } catch(err) { alert(err.response?.data?.error || "Error"); }
  };

  const handleDelete = async (id) => {
    if (!window.confirm("Delete this teacher?")) return;
    await API.delete(`/api/teachers/${id}`);
    fetchTeachers();
  };

  return (
    <div className="layout">
      <Sidebar />
      <div className="main-content">
        <Navbar title="Teachers Management" />
        <div style={{marginBottom:"20px"}}>
          <button className="btn btn-primary" onClick={() => setShowForm(!showForm)}>
            ➕ Add Teacher
          </button>
        </div>

        {showForm && (
          <div className="card" style={{marginBottom:"20px"}}>
            <h3 style={{marginBottom:"16px",fontWeight:"700",color:"#4c1d95"}}>Add New
Teacher</h3>
            <form onSubmit={handleAdd}
```

```
              style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:"12px"}}>
              {[
                ["Full Name","name","text"],
                ["Email","email","email"],
                ["Subject","subject","text"],
                ["Phone","phone","text"],
                ["Qualification","qualification","text"],
              ].map(([label,key,type]) => (
                <div className="form-group" key={key}>
                  <label>{label}</label>
                  <input type={type} value={form[key]}
                    onChange={e=>setForm({...form,[key]:e.target.value})} />
                </div>
              ))}
              <div style={{gridColumn:"1/-1",display:"flex",gap:"10px"}}>
                <button type="submit" className="btn btn-primary">Save</button>
                <button type="button" className="btn btn-outline"
                  onClick={()=>setShowForm(false)}>Cancel</button>
              </div>
            </form>
          </div>
        )}

        <div className="card">
          {loading ? <Spinner /> : (
            <div className="table-wrapper">
              <table>
                <thead><tr>
                  <th>Name</th><th>Email</th><th>Subject</th>
                  <th>Phone</th><th>Qualification</th><th>Actions</th>
                </tr></thead>
                <tbody>
                  {teachers.map(t => (
                    <tr key={t.id}>
                      <td>{t.name}</td>
                      <td>{t.email}</td>
                      <td>{t.subject}</td>
                      <td>{t.phone}</td>
                      <td>{t.qualification}</td>
                      <td>
                        <button className="btn btn-danger"
                          style={{padding:"6px 12px",fontSize:"12px"}}
                          onClick={() => handleDelete(t.id)}>
                          🗑
                        </button>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          )}
        </div>
      </div>
    </div>
  );
};

export default Teachers;
```

SECTION 23: DEPLOYMENT GUIDE

23.1 Backend Deployment on Railway

Step 1: Go to https://railway.app and sign up
Step 2: Click "New Project" → "Deploy from GitHub"
Step 3: Connect your GitHub repository with Spring Boot code
Step 4: Add MySQL plugin to Railway project
Step 5: Set environment variables in Railway:

```
SPRING_DATASOURCE_URL=jdbc:mysql://...(Railway gives this)
SPRING_DATASOURCE_USERNAME=root
SPRING_DATASOURCE_PASSWORD=...(Railway gives this)
SPRING_MAIL_USERNAME=youremail@gmail.com
SPRING_MAIL_PASSWORD=your_app_password
```

23.2 Gmail App Password Setup

To use Gmail for OTP:

- Go to Google Account → Security → 2-Step Verification (enable it)
- Then go to "App passwords"
- Create an app password for "Mail"
- Copy the 16-character password and use it in application.properties

23.3 Frontend Deployment on Vercel

Step 1: Go to https://vercel.com and sign up
Step 2: Import your GitHub repository with React code
Step 3: Add environment variable in Vercel:

```
VITE_API_URL=https://your-railway-backend-url.up.railway.app
```

Step 4: Deploy and get your Vercel URL

23.4 Update CORS for Production

In CorsConfig.java, add your Vercel URL:

```
config.addAllowedOrigin("https://your-project.vercel.app");
```

23.5 Build Frontend for Production

```
cd eduflow-frontend
npm run build
# This creates a "dist" folder - Vercel handles this automatically
```

SECTION 24: TESTING APIS WITH POSTMAN

24.1 Download and Setup Postman

Download from https://www.postman.com/downloads/
Create a new collection named "EduFlow APIs"

24.2 Test Auth APIs

Send OTP:

```
Method: POST
URL: http://localhost:8080/api/auth/send-otp?email=test@gmail.com
Body: (none)
```

Register:

```
Method: POST
URL: http://localhost:8080/api/auth/register?otp=123456
Body (JSON):
{
  "name": "John Doe",
  "email": "test@gmail.com",
  "password": "test123",
  "role": "STUDENT"
}
```

Login:

```
Method: POST
URL: http://localhost:8080/api/auth/login
Body (JSON):
{
  "email": "test@gmail.com",
  "password": "test123"
}
```

24.3 Test Student APIs

```
GET  http://localhost:8080/api/students
POST http://localhost:8080/api/students
Body: { "name":"Jane","email":"jane@gmail.com","rollNumber":"S001" }

DELETE http://localhost:8080/api/students/1
GET    http://localhost:8080/api/students/search?name=jane
```

24.4 Test Attendance & Marks

```
POST http://localhost:8080/api/attendance
Body: { "studentId":1, "date":"2024-01-15", "status":"PRESENT", "subject":"Math" }

POST http://localhost:8080/api/marks
Body: { "studentId":1, "subject":"Math", "marksObtained":85, "totalMarks":100,
"examType":"FINAL", "semester":"1" }

GET  http://localhost:8080/api/analytics/student/1
GET  http://localhost:8080/api/pdf/marksheet/1
```

SECTION 25: QUICK START CHECKLIST

Backend Setup Checklist

- ✅ Create Spring Boot project at start.spring.io
- ✅ Add OpenPDF dependency in pom.xml
- ✅ Configure application.properties (DB + Email)
- ✅ Create MySQL database: eduflow_db
- ✅ Create all model files (Section 3)
- ✅ Create all DTO files (Section 4)
- ✅ Create all repository files (Section 5)
- ✅ Create CorsConfig.java and SecurityConfig.java (Section 6)
- ✅ Create GlobalExceptionHandler.java (Section 7)
- ✅ Create EmailService.java and OtpService.java (Section 8)
- ✅ Create AuthService.java and AuthController.java (Section 9)
- ✅ Create StudentService.java and StudentController.java (Section 10)
- ✅ Create Attendance + Marks files (Section 11)
- ✅ Create AnalyticsService.java and AnalyticsController.java (Section 12)
- ✅ Create PdfService.java and PdfController.java (Section 13)
- ✅ Create TeacherService.java and TeacherController.java (Section 14)
- ✅ Run project in Eclipse → Check console for errors

Frontend Setup Checklist

- ✅ Create Vite React project
- ✅ Install: npm install axios react-router-dom recharts lucide-react
- ✅ Create .env file with VITE_API_URL
- ✅ Create axiosConfig.js (Section 15)
- ✅ Create AuthContext.jsx (Section 15)
- ✅ Create index.css with purple theme (Section 16)
- ✅ Create Sidebar.jsx, Navbar.jsx, StatCard.jsx, Spinner.jsx, ProtectedRoute.jsx (Section 17)
- ✅ Create Login.jsx, Signup.jsx, ForgotPassword.jsx (Section 18)
- ✅ Create AdminDashboard.jsx, TeacherDashboard.jsx, StudentDashboard.jsx (Section 19)
- ✅ Create Students.jsx, Attendance.jsx, Marks.jsx (Section 20)
- ✅ Create Analytics.jsx with charts (Section 21)
- ✅ Create Teachers.jsx (Section 22)
- ✅ Update App.jsx with all routes (Section 15)
- ✅ Run: npm run dev → Open http://localhost:5173

Default Passwords

When teacher/admin adds a student via API:

- Student default password: student@123
- Teacher default password: teacher@123

📌 *Students/Teachers should change password via forgot-password flow after first login.*

Common Errors & Fixes

Error: CORS issue
Fix: Check CorsConfig.java has correct React URL (http://localhost:5173)

Error: Cannot connect to database
Fix: Check MySQL is running, database eduflow_db exists, credentials are correct in application.properties

Error: Email not sending
Fix: Use Gmail App Password (not your normal Gmail password), enable 2FA first

Error: OpenPDF class not found
Fix: Add OpenPDF dependency in pom.xml and do Maven → Update Project in Eclipse

---

🎉 **Congratulations! Your EduFlow project is complete!**

Built with ❤️ | Spring Boot + React + MySQL